

WinniKnight: MindGuardians

Documentacion de Integracion Funcional — Web

Entrega 4 — Paso 4 | Fundamentos de Aplicaciones Web | Grupo 3

Integrante	Carne	Correo
Pablo Urbina	24001508	24001508@galileo.edu
Angel Camargo	24003664	angel.camargo@galileo.edu

1. Descripcion del Proyecto

WinniKnight: MindGuardians es una plataforma web construida con Next.js 14 que complementa la app Android del mismo proyecto. Comparte la misma base de datos Firebase con la app Android — los datos del heroe son sincronizados entre ambas plataformas en tiempo real.

2. Arquitectura del Sistema

El sistema web sigue una arquitectura de tres capas:

- Navegador (Next.js Client) — interfaz de usuario, Firebase Auth y Firestore
- Servidor Next.js (API Routes) — proxy seguro hacia la Cloud Function
- Firebase Cloud Functions (Python) — backend que protege la API key de Gemini

La URL de la Cloud Function y la API key de Gemini nunca llegan al navegador. Viven en variables de entorno del servidor.

2.1 Flujo del Oraculo

```
Navegador -> POST /api/oracle (Next.js servidor)
          -> Cloud Function (Python)
          -> Gemini API -> respuesta epica -> navegador
```

3. Stack Tecnológico

Capa	Tecnologia
Framework	Next.js 14.2.5
Lenguaje	TypeScript
UI	React 18 + Tailwind CSS
Graficas	Recharts 2.12.7
Iconos	Lucide React 0.383.0
Autenticacion	Firebase Auth
Base de datos	Firebase Firestore
Backend IA	Next.js API Routes + Cloud Functions (Python)
IA	Gemini API (gemini-2.5-pro)

4. Dependencias (package.json)

Libreria	Version	Uso
next	14.2.5	Framework principal
react / react-dom	18.3.1	UI
firebase	10.14.1	Auth + Firestore
recharts	2.12.7	Graficas de estadisticas
lucide-react	0.383.0	Iconos
tailwindcss	3.4.6	Estilos
typescript	5.5.3	Tipado estatico

5. Archivos Creados y Modificados

5.1 Archivos nuevos creados

Archivo	Descripcion
src/lib/firebase.ts	Inicializacion de Firebase (Auth + Firestore)
src/lib/auth.ts	Funciones de autenticacion: login, register, logout
src/lib/firestore.ts	Funciones de lectura/escritura en Firestore
src/app/login/page.tsx	Pantalla de inicio de sesion
src/app/register/page.tsx	Pantalla de registro con validaciones
src/app/api/oracle/route.ts	API Route: proxy seguro hacia Cloud Function
.env.local	Variables de entorno (no se sube al repositorio)

5.2 Archivos modificados

Archivo	Cambio realizado
package.json	Agregada dependencia firebase 10.14.1
src/app/page.tsx	Verificacion de sesion, carga de datos reales de Firestore, redireccion a /login si no hay sesion
src/components/oracle/GeminiOracle.tsx	handleSend conectado a /api/oracle, estados de carga y error

6. Variables de Entorno

Archivo .env.local en la raiz del proyecto. Nunca se sube al repositorio.

Variable	Acceso	Descripcion
NEXT_PUBLIC_FIREBASE_API_KEY	Cliente	API key del proyecto Firebase
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	Cliente	Dominio de autenticacion Firebase
NEXT_PUBLIC_FIREBASE_PROJECT_ID	Cliente	ID del proyecto Firebase
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	Cliente	Bucket de Storage Firebase
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	Cliente	Sender ID para notificaciones
NEXT_PUBLIC_FIREBASE_APP_ID	Cliente	App ID de la app web en Firebase
ORACLE_FUNCTION_URL	Servidor	URL de la Cloud Function del Oraculo

El prefijo NEXT_PUBLIC_ hace que la variable sea accesible en el cliente. ORACLE_FUNCTION_URL no lo lleva porque solo existe en el servidor — el navegador nunca ve esa URL.

7. Estructura de Firestore

Identica a la de la app Android. Ambas plataformas comparten los mismos documentos.

```
users/  
  {uid} /  
    displayName: String  
    email:      String  
    heroLevel:  Number  
    heroXp:     Number  
    heroGold:   Number  
    heroHp:     Number
```

```
totalXp:      Number
battles/
  {battleId}/
    date:      Timestamp
    habitType: String
    result:     String
    goldEarned: Number
    xpEarned:  Number
```

8. Flujos Funcionales

8.1 Flujo de Autenticacion

- Usuario entra a / — page.tsx verifica sesión con onAuthStateChanged
- Sin sesion activa: redirige automaticamente a /login
- Login exitoso: redirige a /
- Sin cuenta: navega a /register
- Registro: valida nombre de héroe único en Firestore, crea usuario en Auth, guarda displayName, crea documento Firestore con stats iniciales
- Redirige a /login tras registro exitoso

8.2 Flujo de Carga de Datos

- onAuthStateChanged detecta sesion activa
- loadUserData() lee users/{uid} de Firestore
- page.tsx actualiza estado: displayName, heroLevel, heroXp, heroGold, heroHp
- TopBar, WelcomeBanner y Sidebar muestran datos reales del usuario
- El resto de componentes siguen usando datos mock hasta la entrega final

8.3 Flujo del Oraculo (Gemini AI)

- Usuario escribe hazana de salud y presiona enviar
- GeminiOracle.tsx hace POST a /api/oracle con el mensaje
- route.ts (servidor Next.js) recibe el mensaje
- route.ts hace POST a la Cloud Function con ORACLE_FUNCTION_URL
- Cloud Function llama a Gemini API con el prompt RPG
- Respuesta epica regresa al chat del usuario

9. Configuracion e Instalacion

9.1 Requisitos

- Node.js 18+

- Cuenta de Firebase con proyecto MindGuardians configurado
- Cloud Function desplegada (compartida con Android)

9.2 Pasos

```
npm install
# Crear .env.local con las variables de Firebase y URL de Cloud Function
npm run dev
# Abrir http://localhost:3000
```

10. Seguridad

- La API key de Gemini vive en Firebase Secrets, nunca en el código fuente ni en el navegador
- ORACLE_FUNCTION_URL vive en .env.local sin prefijo NEXT_PUBLIC_, el navegador nunca la ve
- .env.local esta en .gitignore y nunca se sube al repositorio
- Firebase Authentication maneja el ciclo de vida de sesiones
- page.tsx redirige a /login si no hay sesión activa, protegiendo el dashboard
- Firestore en modo Test durante desarrollo — reglas de seguridad en entrega final

11. Sincronización con Android

Ambas plataformas comparten el mismo proyecto Firebase (mindguardians-b07d3) y la misma colección users en Firestore:

- Un usuario registrado en Android puede iniciar sesión en la web con las mismas credenciales
- El progreso del héroe (level, gold, xp) se refleja en ambas plataformas
- Las batallas guardadas desde Android aparecen en el historial web
- El ranking global es compartido entre ambas plataformas

12. Pantallas y Componentes

12.1 Pantallas

Ruta	Descripción
/login	Inicio de sesión con email y contraseña
/register	Registro con nombre de héroe único, email y contraseña
/	Dashboard principal con datos reales de Firestore

12.2 Componentes del Dashboard

Componente	Descripcion
WelcomeBanner	Mensaje de bienvenida con nombre real del usuario y XP
VitalityStats	Grafica de barras semanal con Recharts
GeminiOracle	Chat en tiempo real con Gemini AI via Cloud Function
HeroesRanking	Tabla de ranking global
EquipmentInventory	Grid de inventario con sistema de raridades
BattlesHistory	Historial filterable de combates por fecha y tipo de habito
GoldShop	Tienda de equipamiento con filtros por categoria